

DOCUMENTO DE VISÃO DE PROJETO

NEXUS GEAR

Plataforma de e-commerce de periféricos gamer

Documento de Visão de Projeto

Nexus Gaming Gear



Faculdade de Tecnologia de Mogi das Cruzes

Tecnologia em Análise e Desenvolvimento de Sistemas | Tecnologia em Agronegócio | Tecnologia em Recursos Humanos



Histórico de Versões

Data	Versão	Descrição	Autor	Revisor
12/09/2026	1.0	Elaboração inicial: necessidades de negócio, escopo preliminar, requisitos de alto nível, arquitetura e visões do sistema	Equipe Nexus Gaming Gear	Rodrigo Rocha

Cliente FATEC Mogi das Cruzes – Interno (projeto acadêmico da disciplina Laboratório de Engenharia de Software)

Documento Documento de Visão de Projeto: Nexus Gaming Gear

Data 12 de setembro de 2026

Autor Equipe Nexus Gaming Gear
Lucas Carmona · Fernando Camargo
<https://github.com/lucasscarmona/nexus-gaming-ecommerce>

Página de Assinaturas

Revisado e
Aprovado por: Rodrigo Rocha

____/____/____



Índice

Objetivo	4
Escopo	4
Referências	4
Necessidades de Negócio	4
Objetivo do Projeto	5
Declaração Preliminar de Escopo	5
Descrição	5
Produtos a serem entregues	6
Requisitos	6
Requisitos Funcionais	6
Requisitos Não Funcionais	7
Cronograma de Marcos Sumariado	7
Regras de Negócio	7
Premissas	8
Influência das Partes Interessadas	8
Representação Arquitetural	9
Restrições Arquiteturais	11
Objetivos e Restrições Arquiteturais	12
Visão de Use Case	12
Diagrama de caso de uso geral do sistema	14
Descrição dos Casos de Uso Arquiteturalmente Significativos	14
Visão de Lógica	16
Camada de Apresentação	18
Camada de Negócio	18
Pacote Controller	19
Pacote Model	20
Camada de Persistência	22
Realização dos Casos de Uso Significativos	23
Visão de Implantação	24
Visão de Implementação	25
Visão de Dados	26
Tamanho e Performance	30
Qualidade	30
Cronograma Macro	31
Referências bibliográficas	32



1. Objetivo

Este documento formaliza o início do projeto Nexus Gaming Gear, uma plataforma de comércio eletrônico de periféricos gamer desenvolvida pela equipe ao longo da disciplina. Ele registra o problema que motivou a criação do sistema, o entendimento atual das necessidades do negócio e uma descrição resumida da solução que será construída para atendê-las.

O documento apresenta também a visão arquitetural da solução, reunindo as decisões técnicas já tomadas pela equipe sobre organização em camadas, tecnologias e estrutura de dados. Seu propósito é alinhar as expectativas do professor orientador e dos integrantes da equipe antes que a implementação avance, servindo de referência para o Documento de Requisitos e para o planejamento das entregas seguintes.

1.1. Escopo

Este documento trata da visão inicial do Nexus Gaming Gear: um sistema web que permitirá a uma loja de periféricos gamer vender seus produtos pela internet, controlar o ciclo de vida dos pedidos e administrar trocas, cupons e estoque. São apresentados aqui o escopo preliminar do produto, os requisitos de alto nível já conhecidos e as visões arquiteturais que orientarão o desenvolvimento.

Não fazem parte deste documento o detalhamento dos requisitos (fluxos principais e alternativos, regras de negócio completas e critérios de aceitação), o dicionário de dados e o projeto detalhado das classes. Esses itens serão tratados no Documento de Requisitos e na modelagem de banco de dados, nas etapas seguintes da disciplina.

1.2. Referências

Para a construção deste documento foram utilizadas as seguintes fontes:

- Reuniões de alinhamento da equipe realizadas durante as aulas, nas quais foram definidos o domínio do sistema e o escopo da primeira etapa;
- Orientações do professor da disciplina sobre o escopo mínimo esperado para o sistema e sobre os prazos de entrega;
- Protótipo de interface do Nexus Gaming Gear desenvolvido pela própria equipe (arquivo index.html do repositório), usado para validar as funcionalidades previstas;
- Repositório do projeto no GitHub: <https://github.com/lucasscarmona/nexus-gaming-ecommerce>;
- Documentação oficial do Spring Boot, do Spring Data JPA e do PostgreSQL, consultada para as decisões de arquitetura e de persistência.

Este documento influencia os seguintes documentos:

- Documento de Requisitos do Nexus Gaming Gear;
- Modelagem de banco de dados (modelos lógico e físico) e script de criação das tabelas;
- Planejamento e cronograma detalhado das entregas da disciplina.

2. Necessidades de Negócio

Lojas especializadas em periféricos gamer atendem um público que pesquisa configurações, compara especificações técnicas e compra predominantemente pela internet. Uma loja que vende apenas presencialmente ou por canais informais, como mensagens em redes sociais, depende de um vendedor para



cada atendimento: o cliente não consegue consultar preço e disponibilidade sozinho, os pedidos são anotados manualmente e o controle do que já foi vendido, pago e enviado fica espalhado entre planilhas e conversas.

Esse arranjo gera três problemas concretos. O primeiro é a perda de vendas fora do horário comercial e sempre que o cliente não recebe resposta rápida. O segundo é o acompanhamento do pedido, que depende de o cliente perguntar por mensagem em que etapa ele está: isso consome tempo da equipe e gera insegurança em compras de valor elevado, comuns nessa categoria de produto. O terceiro são as trocas e devoluções, frequentes em periféricos por defeito de fabricação ou por incompatibilidade, que hoje não têm registro estruturado. Fica difícil saber quantas peças voltaram, por qual motivo e qual crédito cada cliente recebeu.

Há ainda a dificuldade de enxergar o próprio negócio. Sem registro digital das vendas, o responsável pela loja não consegue responder quais produtos vendem mais, qual foi o faturamento do período ou quanto do estoque está comprometido com pedidos ainda não enviados. Enquanto não existir um sistema que centralize catálogo, pedidos, pagamentos, trocas e estoque, esses problemas continuarão a ocorrer e limitarão o crescimento da loja.

3. Objetivo do Projeto

Desenvolver o Nexus Gaming Gear, um sistema web de comércio eletrônico que permita à loja vender periféricos gamer pela internet e acompanhar cada venda do início ao fim. O sistema deve:

- manter o catálogo de produtos e o controle de estoque, exibindo ao cliente apenas itens realmente disponíveis;
- permitir que o cliente crie sua conta, com vários endereços de entrega e vários cartões de crédito cadastrados;
- conduzir a compra do carrinho até a confirmação do pedido, com aplicação de cupons e pagamento dividido entre mais de um cartão;
- controlar o ciclo de vida do pedido por meio de estados bem definidos, do processamento à entrega, registrando cada mudança em histórico;
- registrar solicitações de troca e a análise dessas solicitações, gerando cupom de crédito ao cliente quando a troca for autorizada;
- oferecer ao administrador um painel com o andamento dos pedidos, a base de clientes e a análise das vendas do período.

Espera-se, com isso, que a loja passe a vender sem depender de atendimento individual, que o cliente acompanhe o próprio pedido sem precisar acionar a equipe e que as informações de vendas, trocas e estoque fiquem registradas em um único lugar, servindo de base para as decisões do negócio.

4. Declaração Preliminar de Escopo

Esta seção apresenta, em alto nível, o que será entregue no projeto. O escopo aqui descrito é preliminar: reflete o entendimento da equipe no início do desenvolvimento e será refinado no Documento de Requisitos, podendo sofrer ajustes conforme as prioridades definidas com o professor ao longo da disciplina.

4.1. Descrição

O Nexus Gaming Gear é uma loja virtual de periféricos gamer: teclados, mouses, headsets, mousepads e acessórios. O visitante navega pelo catálogo e consulta as especificações dos produtos; ao criar uma conta, torna-se cliente e passa a montar seu carrinho, escolher entre os endereços de entrega que cadastrou, aplicar cupons promocionais ou de troca e pagar o pedido com um ou mais cartões de crédito. Depois de comprar, acompanha o pedido pela área do cliente, podendo cancelá-lo enquanto ainda estiver em processamento, confirmar o recebimento e solicitar a troca de itens já entregues. Quem entra com uma conta de nível administrativo vê, no mesmo site, um botão a mais, que leva à página de administração. É por ela que a loja cadastra produtos, controla o estoque, movimenta o status dos pedidos, analisa as solicitações de troca e



acompanha o desempenho das vendas. Um assistente virtual integrado à loja sugere produtos complementares durante a navegação.

4.2. Produtos a serem entregues

Os seguintes itens são considerados produtos do projeto em sua primeira etapa:

- Sistema Nexus Gaming Gear implementado (código-fonte e aplicação executável), contemplando cadastro de clientes, catálogo, carrinho, fechamento de pedido, controle de status, trocas, cupons e página de administração;
- Este Documento de Visão de Projeto e o Documento de Requisitos do sistema;
- Modelos de dados lógico e físico da base `nexus_gaming_db`, com o script de criação das tabelas;
- Diagramas UML do sistema: caso de uso, camadas e pacotes, classes de domínio, sequência e implantação;
- Repositório versionado no GitHub, com o histórico de commits da equipe, e instruções de execução do ambiente local (Java 21, Maven e PostgreSQL);
- Apresentação final do sistema em funcionamento para o professor e a turma.

4.3. Requisitos

Os requisitos a seguir representam o que a equipe já conhece do sistema no início do projeto e estão escritos em alto nível. Fluxos alternativos, exceções, validações campo a campo e critérios de aceitação serão detalhados no Documento de Requisitos.

4.3.1. Requisitos Funcionais

O sistema deve ser capaz de cadastrar clientes, registrando nome, CPF, data de nascimento, telefone, e-mail e senha de acesso.

O sistema deve ser capaz de manter vários endereços de entrega e de cobrança e vários cartões de crédito para um mesmo cliente.

O sistema deve ser capaz de permitir que o cliente altere seus dados cadastrais e inative sua conta, preservando o histórico dos pedidos já realizados.

O sistema deve ser capaz de manter o catálogo de produtos, com categoria, descrição, preço e situação (ativo ou inativo).

O sistema deve ser capaz de controlar a quantidade disponível e a quantidade reservada de cada produto em estoque.

O sistema deve ser capaz de exibir o catálogo ao visitante e permitir a consulta de produtos por categoria e por nome.

O sistema deve ser capaz de montar um carrinho de compras, permitindo incluir itens, alterar quantidades e remover itens antes do fechamento.

O sistema deve ser capaz de finalizar o pedido com a seleção do endereço de entrega, o cálculo do valor total e o registro dos itens comprados.

O sistema deve ser capaz de aplicar cupons promocionais e cupons de troca no fechamento do pedido, abatendo o valor correspondente.

O sistema deve ser capaz de registrar o pagamento de um pedido dividido entre um ou mais cartões de crédito do cliente.

O sistema deve ser capaz de controlar o ciclo de vida do pedido nos estados em processamento, pagamento aprovado, pagamento recusado, em transporte, entregue e cancelado, registrando o histórico de cada mudança.

O sistema deve ser capaz de permitir que o cliente acompanhe seus pedidos, cancele pedidos ainda não enviados e confirme o recebimento.

O sistema deve ser capaz de registrar solicitações de troca de itens entregues e permitir que o administrador as autorize ou recuse.



O sistema deve ser capaz de gerar automaticamente um cupom de troca em favor do cliente quando uma solicitação de troca for autorizada.

O sistema deve ser capaz de devolver ao estoque as unidades correspondentes a trocas autorizadas e a pedidos cancelados.

O sistema deve ser capaz de oferecer ao administrador a alteração do status dos pedidos e a consulta à base de clientes.

O sistema deve ser capaz de apresentar ao administrador o faturamento do período e a evolução das vendas em forma de gráfico.

O sistema deve ser capaz de sugerir produtos complementares ao cliente durante a navegação, por meio do assistente virtual da loja.

O sistema deve ser capaz de autenticar os usuários e de guardar o nível de acesso de cada conta, que pode ser cliente ou administrador.

O sistema deve ser capaz de exibir o botão de acesso à página de administração apenas para contas de nível administrativo, conferindo essa permissão também no servidor a cada requisição.

4.3.2. Requisitos Não Funcionais

Desenvolver o back-end na linguagem Java, versão 21, com o framework Spring Boot 4.1.

Utilizar o PostgreSQL como sistema gerenciador de banco de dados, com acesso por meio do Spring Data JPA e do Hibernate.

Construir a interface do usuário em HTML5, Tailwind CSS e JavaScript, consumindo a API do sistema.

Organizar a solução em camadas (apresentação, negócio, persistência e integração), com o back-end expondo uma API REST em formato JSON.

Armazenar as senhas de forma cifrada e não registrar o número completo dos cartões de crédito, guardando apenas bandeira, nome impresso e os quatro últimos dígitos.

Utilizar o Apache Maven para build e gerenciamento de dependências e o Git/GitHub para versionamento do código.

Responder às consultas ao catálogo em até três segundos em condições normais de uso.

Apresentar interface responsiva, adaptada a telas de computador e de celular.

O sistema deve funcionar nos seguintes navegadores:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

4.3.3. Cronograma de Marcos Sumariado

Os marcos de entrega acompanham o calendário da disciplina no semestre 2026/2: entrega deste Documento de Visão de Projeto, entrega do Documento de Requisitos, entrega das versões incrementais do sistema ao final de cada sprint e apresentação final. As datas previstas estão detalhadas na seção Cronograma Macro deste documento.

4.3.4. Regras de Negócio

- Somente clientes autenticados e com conta ativa podem finalizar pedidos.
- O pedido pode ser cancelado pelo próprio cliente enquanto estiver em processamento ou com pagamento aprovado; depois de entrar em transporte, o cancelamento depende de análise do administrador.
- O valor de um pedido pode ser dividido entre vários cartões, desde que a soma dos valores informados seja igual ao total a pagar.
- Cupons de troca são nominais: só podem ser usados pelo cliente que os recebeu e apenas uma vez. Cupons promocionais podem ter data de validade e valor mínimo de compra.
- A troca só pode ser solicitada para itens de pedidos com status entregue e dentro do prazo definido pela loja, contado a partir da confirmação de recebimento.
- A autorização de uma troca gera um cupom de crédito no valor dos itens devolvidos e retorna as unidades correspondentes ao estoque.



- O estoque dos itens é reservado no fechamento da compra e baixado definitivamente quando o pedido entra em transporte; o cancelamento libera a reserva.
- A inativação da conta preserva os pedidos já realizados, que continuam disponíveis para consulta administrativa.

5. Premissas

As premissas a seguir são condições que a equipe considera verdadeiras para planejar o projeto. Caso alguma delas não se confirme, o escopo ou o cronograma precisarão ser revistos.

- O projeto é orientado pelo(a) professor(a) Rodrigo Rocha, responsável pela disciplina, que atua também como cliente do sistema, validando escopo e entregas.
- Não há cliente externo real: as regras de negócio da loja são definidas pela própria equipe, com base em lojas virtuais existentes, e validadas com o professor.
- Os integrantes têm disponibilidade para se reunir semanalmente e dispõem, em suas máquinas, do ambiente necessário ao desenvolvimento (Java 21, Maven, PostgreSQL e Git).
- O repositório no GitHub é a única fonte oficial do código e da documentação do projeto.
- Os dados usados em testes são fictícios, criados pela equipe; nenhum dado real de cliente ou de cartão de crédito será utilizado.
- A integração com o gateway de pagamento será simulada nesta etapa, sem contrato com uma operadora real; a interface de integração, porém, será projetada de modo a permitir a substituição por um serviço real.
- O prazo do projeto é o do semestre letivo, e o escopo priorizado cabe nesse período considerando a carga horária disponível dos integrantes.
- O sistema será executado em ambiente local durante o desenvolvimento; havendo hospedagem, ela será feita em serviço gratuito de nuvem.

6. Influência das Partes Interessadas

Os interessados a seguir influenciam o andamento e o resultado do projeto. Identificá-los ainda na iniciação permite antecipar prioridades conflitantes e evitar retrabalho.

- **Professor(a) orientador(a) da disciplina:** avalia as entregas e atua como cliente do projeto. É o interessado de maior influência, pois define o escopo mínimo aceitável, os prazos e os critérios de qualidade do documento e do sistema.
- **Equipe de desenvolvimento:** responsável por todas as decisões técnicas e pela construção do sistema. Interessada em entregar um sistema funcional dentro do prazo e em aprender as tecnologias adotadas.
- **Integrante responsável pelo repositório e pela integração do código:** garante que o trabalho de todos seja integrado sem conflitos e que a versão entregue esteja funcionando. Influencia diretamente o ritmo das entregas.
- **Integrante responsável pela modelagem de dados:** interessado em que as entidades e os relacionamentos estejam estáveis antes do avanço da implementação, já que mudanças no modelo impactam todas as camadas.
- **Lojista (papel de cliente simulado, representado pela equipe):** interessado em vender mais e reduzir o tempo gasto com atendimento manual; espera controle claro de pedidos, trocas e estoque.
- **Clientes finais (usuários simulados da loja):** interessados em comprar com segurança, acompanhar o pedido sem depender de atendimento e resolver trocas rapidamente. Suas expectativas orientam as prioridades de usabilidade.



- **Turma da disciplina:** colegas que assistem à apresentação final e cujas dúvidas ajudam a validar a clareza das decisões tomadas pela equipe.

7. Representação Arquitetural

O Nexus Gaming Gear é construído como uma aplicação web organizada em camadas, com back-end em Java 21 e Spring Boot 4.1 e front-end executado no navegador. A comunicação entre as duas partes é feita por uma API REST em JSON, o que mantém a interface independente das regras de negócio e permite evoluir o front-end sem alterar o servidor. A Figura 1 apresenta essa organização.

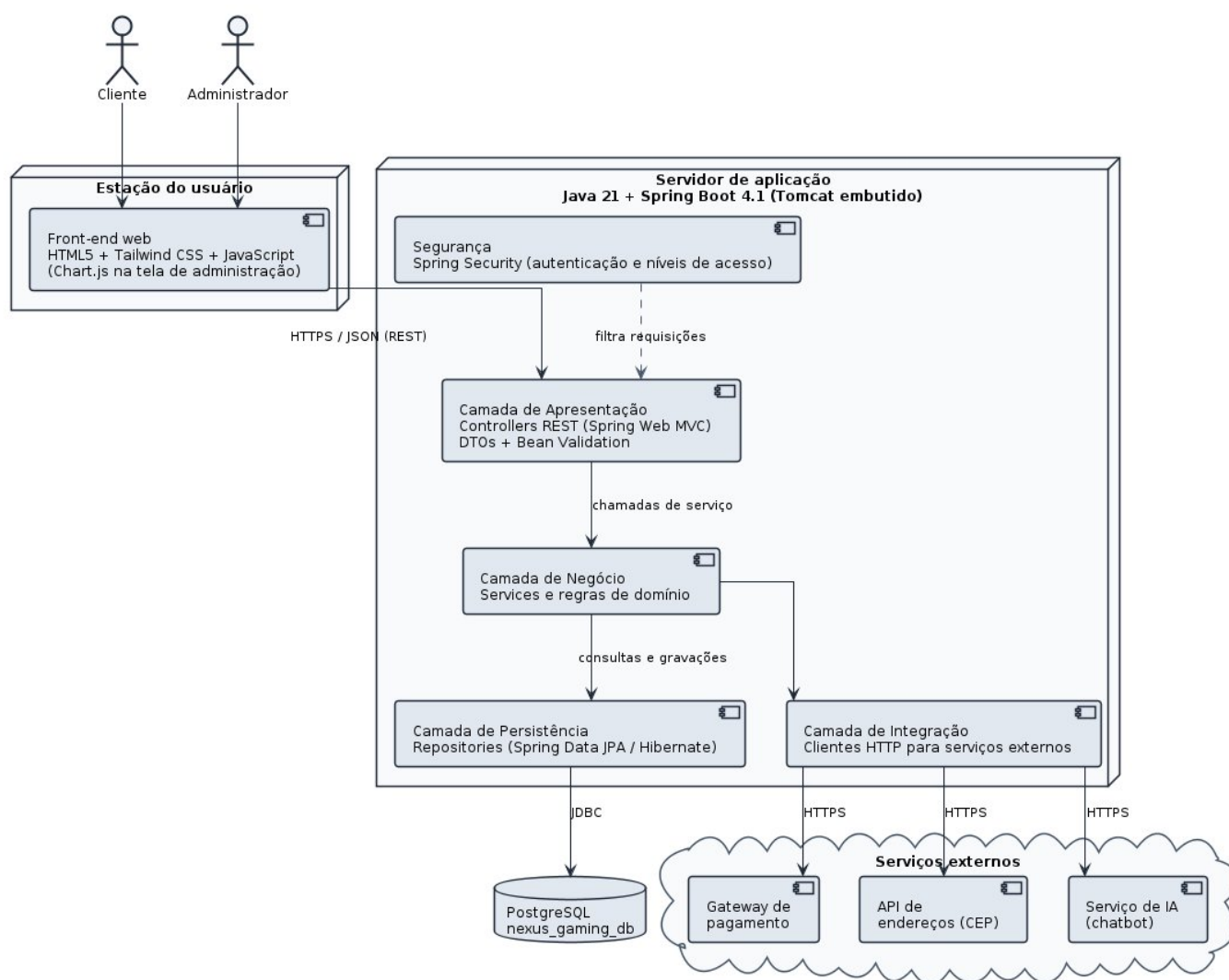


Figura 1 – Representação arquitetural do Nexus Gaming Gear

O **front-end** é uma aplicação web estática (HTML5, Tailwind CSS e JavaScript) servida pelo próprio Spring Boot a partir do diretório resources/static. Ele concentra as telas da loja, do carrinho, da área do cliente e da administração, e obtém todos os dados por chamadas à API. Na tela de administração, a biblioteca Chart.js é usada para apresentar a evolução das vendas.



A **camada de apresentação** do servidor é formada pelos controladores REST, implementados com Spring Web MVC. Eles recebem as requisições, convertem os dados de entrada e saída em objetos de transferência (DTOs), validam o que foi enviado com Bean Validation e delegam a execução à camada de negócio, sem conter regras de negócio.

A **camada de negócio** reúne as classes de serviço e as entidades de domínio. É nela que ficam as regras do sistema: cálculo do valor do pedido, validade e aplicação de cupons, divisão do pagamento entre cartões, transições permitidas do status do pedido e condições para solicitar e autorizar uma troca. O controle transacional também é feito aqui, garantindo que operações compostas, como o fechamento de um pedido, sejam confirmadas por completo ou desfeitas.

A **camada de persistência** é implementada com Spring Data JPA e Hibernate. Os repositórios são interfaces que estendem JpaRepository, e o mapeamento objeto-relacional dispensa a escrita manual de SQL na maior parte das operações. O banco de dados é o PostgreSQL, na base `nexus_gaming_db`, acessado via JDBC.

A **camada de integração** isola o acesso aos serviços externos: o gateway de pagamento, a consulta de endereços por CEP e o serviço de inteligência artificial que apoia o assistente virtual. Cada integração fica encapsulada em uma classe cliente própria, de modo que a troca de provedor afete apenas essa camada. O controle de acesso é transversal e fica a cargo do Spring Security, que autentica os usuários e separa os perfis cliente e administrador.

A Figura 2 detalha essa mesma organização no nível dos pacotes do código-fonte, sob o pacote raiz `com.nexusgaming.ecommerce`.

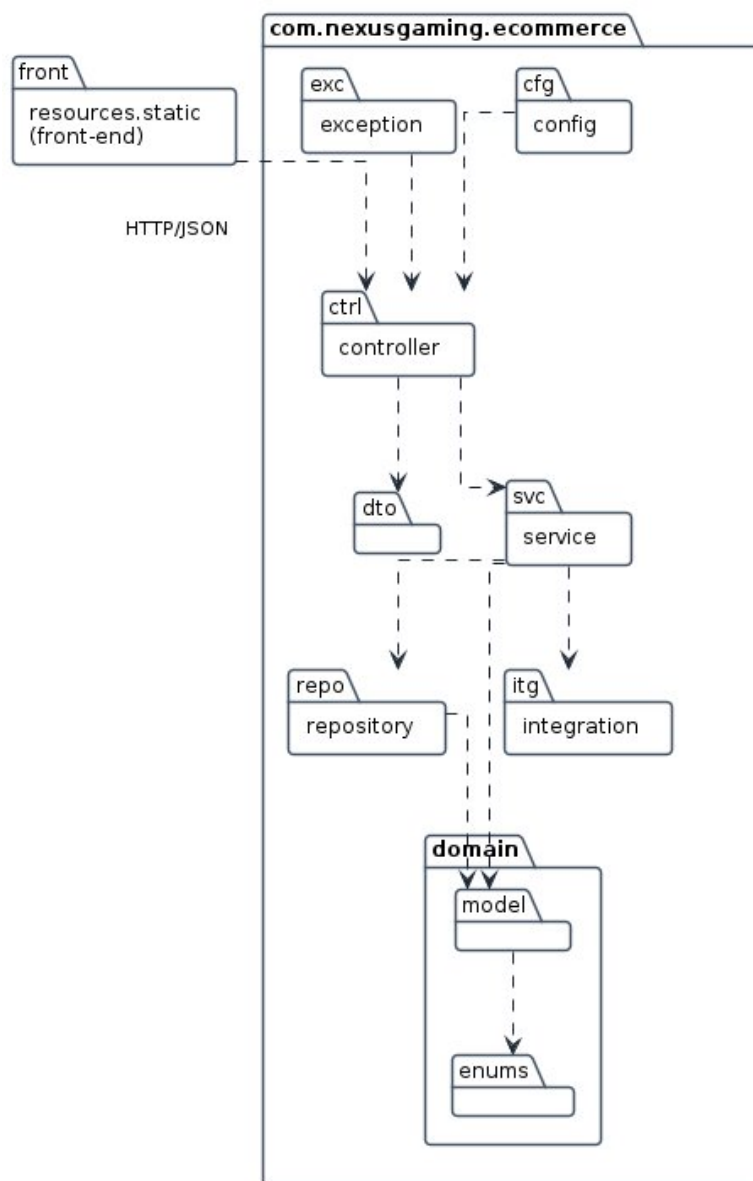


Figura 2 – Diagrama de pacotes do sistema

As dependências entre os pacotes são unidirecionais e sempre da camada mais externa para a mais interna: os controladores conhecem os serviços, os serviços conhecem os repositórios e as entidades de domínio, e nenhum pacote de domínio depende de pacotes de apresentação. Essa regra evita acoplamento circular e permite testar as regras de negócio sem subir a camada web.

7.1. Restrições Arquiteturais

As decisões a seguir já estão tomadas e restringem o desenvolvimento do sistema:

- Linguagem Java na versão 21 (LTS), conforme configurado no projeto Maven;
- Framework Spring Boot 4.1, com os módulos Spring Web MVC, Spring Data JPA, Spring Security e Bean Validation;



- Servidor de aplicação Apache Tomcat embutido no artefato executável, dispensando a instalação de servidor externo;
- SGBD PostgreSQL, com mapeamento objeto-relacional feito pelo Hibernate;
- Projeto Lombok para redução de código repetitivo nas entidades e nos objetos de transferência;
- Apache Maven como ferramenta de build e Git/GitHub para versionamento;
- Front-end em HTML, Tailwind CSS e JavaScript, sem framework de página única, servido pelo próprio back-end.

7.2. Objetivos e Restrições Arquiteturais

Os requisitos a seguir impactam diretamente as decisões de arquitetura do Nexus Gaming Gear:

- o pagamento de um pedido pode ser dividido entre vários cartões, o que exige tratar cada autorização como uma transação independente e consolidar o resultado antes de confirmar o pedido;
- a comunicação com o gateway de pagamento é externa e pode falhar ou demorar, de modo que a integração precisa prever tempo limite e uma situação de pagamento pendente;
- o status do pedido percorre uma sequência definida de estados, e cada transição deve ser registrada em histórico, o que concentra essa regra em um único ponto do domínio;
- os dados de cartão de crédito não podem ser armazenados integralmente, o que obriga a delegar sua guarda ao gateway e manter no banco apenas bandeira, nome impresso e os quatro últimos dígitos;
- o assistente virtual depende de um serviço externo de inteligência artificial e deve ser opcional: se o serviço estiver indisponível, a loja continua funcionando.

Considerando as premissas definidas para o projeto, aplicam-se ainda estas restrições:

- utilização exclusiva de tecnologias gratuitas ou de código aberto, pela ausência de orçamento no contexto acadêmico;
- comunicação entre front-end e back-end apenas por HTTP/JSON, sem dependência de estado de sessão no servidor além da autenticação;
- execução em ambiente local durante o desenvolvimento, o que limita os testes a volumes pequenos de dados;
- a página de administração faz parte da mesma aplicação, e não de um sistema à parte: o que muda é o nível de acesso da conta, verificado no servidor.

8. Visão de Use Case

Esta seção apresenta os casos de uso do Nexus Gaming Gear que mais influenciam as decisões de arquitetura. Não se trata da lista completa de casos de uso do sistema, que será apresentada no Documento de Requisitos.

A seleção considerou pelo menos um dos critérios a seguir:

- casos de uso que incluem outros casos de uso, concentrando comportamento reutilizado em vários fluxos;



- casos de uso que estendem outros casos de uso, alterando o comportamento padrão em situações específicas;
- casos de uso que dependem de serviços externos ao sistema;
- casos de uso que alteram o estado de entidades centrais do domínio, como o pedido e o estoque.



8.1. Diagrama de caso de uso geral do sistema

A Figura 3 apresenta os atores e os principais casos de uso do sistema. O ator Visitante representa quem navega sem estar autenticado; o Cliente herda suas capacidades e acrescenta as operações de compra e de acompanhamento. O Administrador é o usuário cuja conta tem nível de acesso administrativo e que, por isso, alcança a página de administração. Gateway de pagamento e Serviço de IA são atores externos, acionados pelo sistema durante a execução de alguns casos de uso.

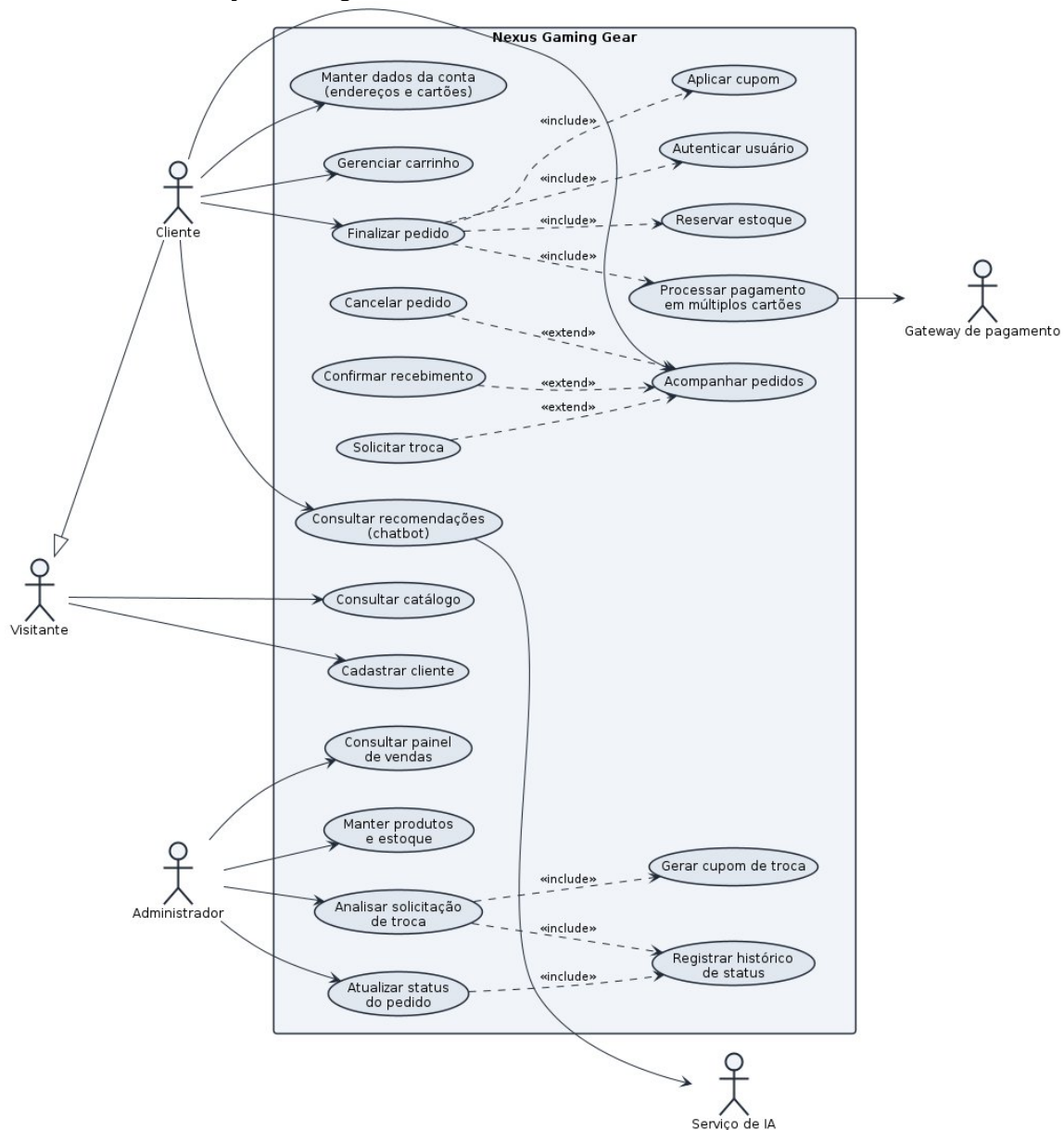


Figura 3 – Diagrama de caso de uso geral do Nexus Gaming Gear

8.2. Descrição dos Casos de Uso Arquiteturalmente Significativos

Finalizar pedido



Inicia quando o cliente autenticado confirma o carrinho e aciona o fechamento da compra. O cliente escolhe o endereço de entrega entre os que cadastrou, informa os cupons que deseja usar e distribui o valor entre um ou mais cartões. O sistema valida os cupons, reserva o estoque dos itens, solicita a autorização de cada cartão ao gateway de pagamento e, se todas forem aprovadas, grava o pedido com status de pagamento aprovado e registra a primeira entrada do histórico. Se alguma autorização for recusada, a reserva de estoque é desfeita e o pedido é gravado como pagamento recusado. É o caso de uso mais significativo do sistema: inclui outros três, altera duas entidades centrais e depende de serviço externo.

Atualizar status do pedido

Inicia quando o administrador seleciona um pedido na página de administração e indica o novo status. O sistema verifica se a transição é permitida a partir do status atual, aplica a mudança, registra autor e data no histórico e, na transição para em transporte, dá baixa definitiva no estoque reservado. Entra na lista porque concentra a máquina de estados do pedido, usada também pelo cancelamento e pela confirmação de recebimento.

Solicitar troca

Estende o acompanhamento de pedidos e inicia quando o cliente escolhe um item de um pedido já entregue e informa a quantidade e o motivo da troca. O sistema verifica se o pedido está entregue e dentro do prazo permitido e registra a solicitação com status de troca solicitada, pendente de análise. Ele pesa na arquitetura por abrir um segundo ciclo de estados no sistema, que corre em paralelo ao do pedido.

Analisar solicitação de troca

Inicia quando o administrador abre uma solicitação pendente na página de administração e decide autorizá-la ou recusá-la. Ao autorizar, o sistema gera um cupom de troca nominal ao cliente, no valor dos itens devolvidos, retorna as unidades ao estoque e atualiza o status da troca; ao recusar, registra o motivo e encerra a solicitação. Inclui a geração de cupom, que também é usada nas ações promocionais da loja.

Manter dados da conta

Inicia quando o cliente acessa a área de cadastro para incluir, alterar ou remover endereços de entrega e de cobrança e cartões de crédito, ou para alterar seus dados pessoais. O sistema valida os dados informados, consulta o serviço externo de CEP para completar o endereço e grava as alterações. Importa para a arquitetura por depender de serviço externo e por alimentar os dados que o fechamento do pedido consome.

Consultar recomendações do assistente virtual

Inicia quando o cliente abre o assistente virtual durante a navegação. O sistema envia ao serviço externo de inteligência artificial o contexto do produto em exibição e devolve ao cliente a sugestão de itens complementares disponíveis em estoque. Se o serviço externo não responder, o assistente é ocultado e a navegação segue normalmente. É o único ponto do sistema que pode ficar indisponível sem derrubar o resto, e essa escolha precisa estar registrada.



Os demais casos de uso, como consultar catálogo, gerenciar carrinho, cadastrar cliente, autenticar usuário, manter produtos e estoque e consultar o painel de vendas, serão descritos em detalhe no Documento de Requisitos.

9. Visão de Lógica

Esta visão apresenta como o Nexus Gaming Gear está organizado internamente em camadas e pacotes, a partir da estrutura real do código-fonte no repositório do projeto, sob o pacote raiz `com.nexusgaming.ecommerce`.

A Figura 4 mostra as camadas do sistema e a direção das dependências entre elas.

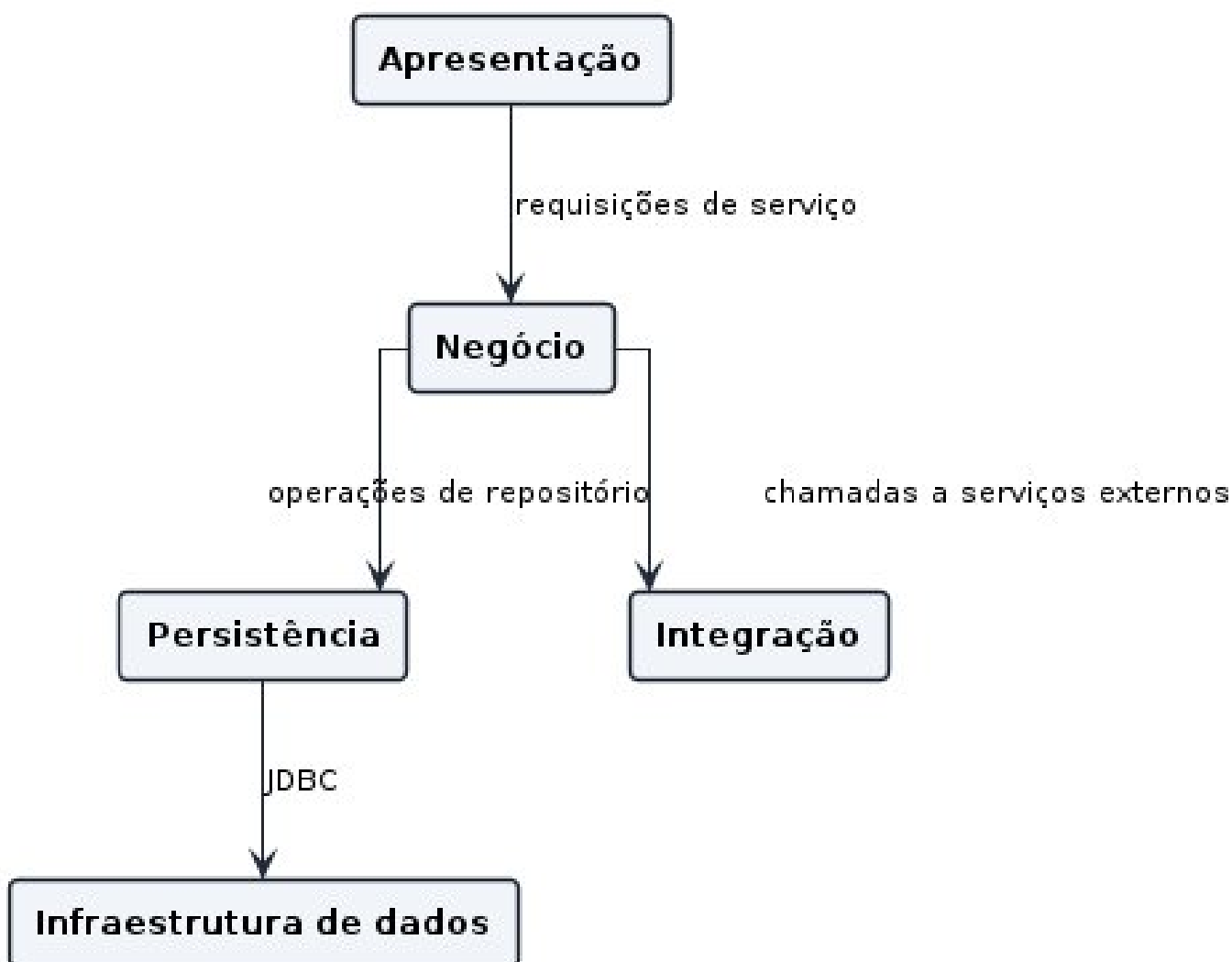


Figura 4 – Diagrama de camadas do Nexus Gaming Gear

Apresentação: contém o front-end executado no navegador e os controladores REST do servidor. Responde pelo contato com o usuário e pela conversão dos dados entre o formato JSON e os objetos da aplicação; não contém regras de negócio.



Negócio: contém as classes de serviço e as entidades de domínio, com as regras do sistema e o controle das transações.

Persistência: contém os repositórios e o mapeamento objeto-relacional responsáveis por gravar e recuperar as entidades no banco de dados.

Integração: contém os clientes dos serviços externos (gateway de pagamento, consulta de CEP e serviço de inteligência artificial) e isola o restante do sistema das particularidades de cada provedor.

A Figura 5 repete essas camadas indicando as tecnologias empregadas em cada uma delas.

Vale destacar que a camada de apresentação se divide entre as duas pontas da aplicação. No navegador ficam o HTML, o CSS e o JavaScript que montam as telas, fazem a verificação prévia dos campos e chamam a API. No servidor ficam os controladores, que recebem essas chamadas, aplicam as validações obrigatórias e delegam o processamento. A verificação feita no navegador existe apenas para melhorar a experiência de uso: nenhuma regra de negócio é decidida ali.

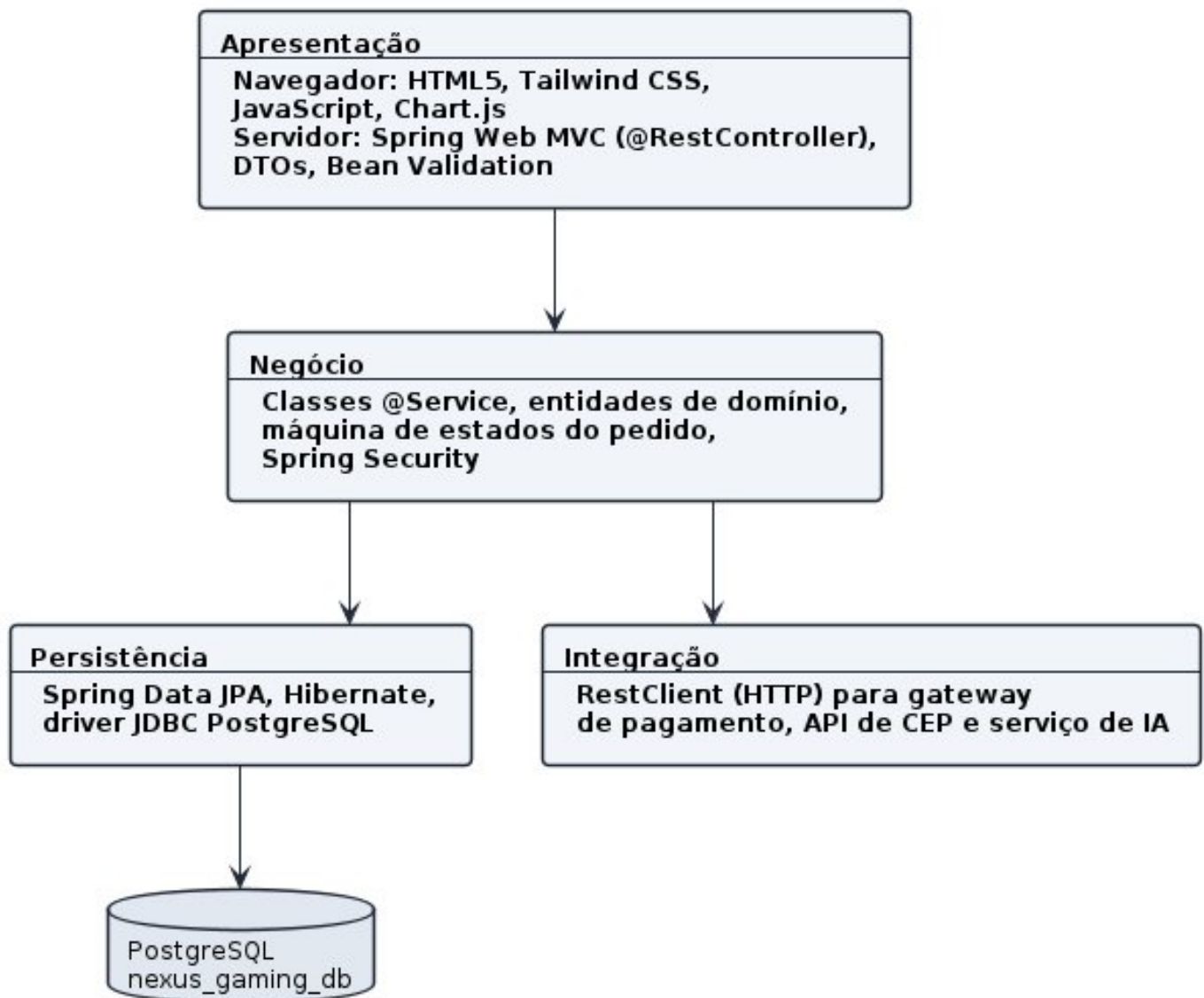


Figura 5 – Camadas do sistema e as tecnologias correspondentes



9.1. Camada de Apresentação

O front-end fica em `resources/static`: a página `index.html` concentra as telas da loja, do carrinho e do checkout, da área do cliente e da administração, alternadas por JavaScript, e os scripts fazem as chamadas à API. O botão que abre a tela de administração só aparece quando a conta autenticada tem nível de acesso administrativo, e o servidor confere essa permissão de novo a cada requisição, já que esconder o botão não protege nada sozinho. No servidor, o pacote `controller` reúne um controlador por área funcional, cada um exposto como `@RestController`. O pacote `dto` contém os objetos de entrada e saída da API, que isolam as entidades de domínio do formato exposto ao cliente, e o pacote `exception` concentra o tratamento de erros, transformando exceções de negócio em respostas HTTP padronizadas. A Figura 6 mostra esses pacotes.

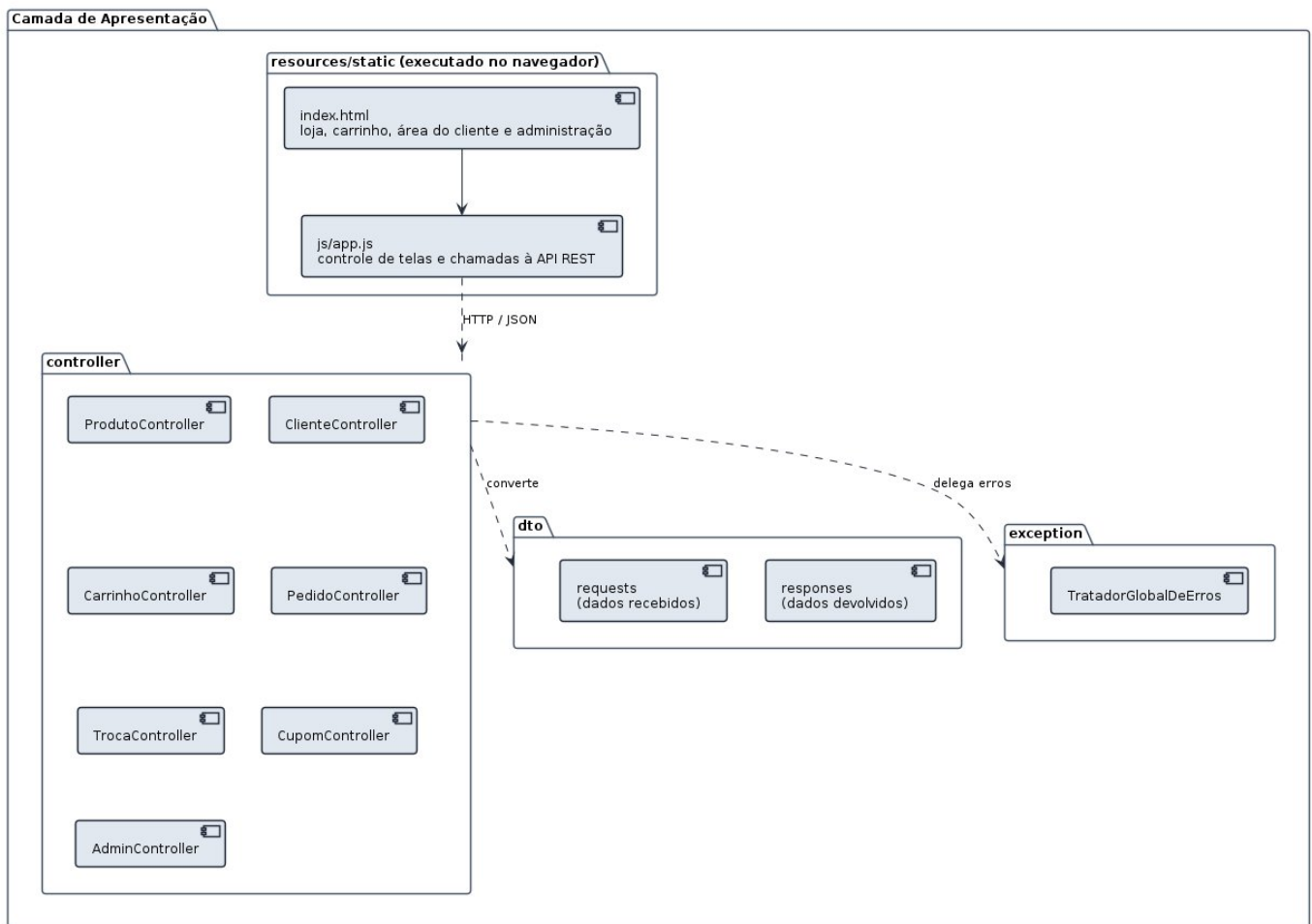


Figura 6 – Pacotes da camada de apresentação

9.2. Camada de Negócio

O pacote `service` contém uma classe de serviço por conjunto de regras: `ClienteService` trata do cadastro e da inativação de contas; `ProdutoService` e `EstoqueService`, do catálogo e da disponibilidade; `CarrinhoService`, da montagem do carrinho; `PedidoService`, do fechamento e do ciclo de vida do pedido; `PagamentoService`, da divisão do pagamento entre cartões; `CupomService`, da validade e da aplicação de cupons; `TrocaService`, do fluxo de trocas; e `RecomendacaoService`, do assistente virtual. O pacote `domain.model` reúne as entidades que representam os conceitos do negócio, e `domain.enums`, os conjuntos fixos de valores, como os status de pedido e de troca. O pacote `config` concentra a configuração de segurança e a separação dos perfis de acesso.



A Figura 7 ilustra os pacotes descritos.

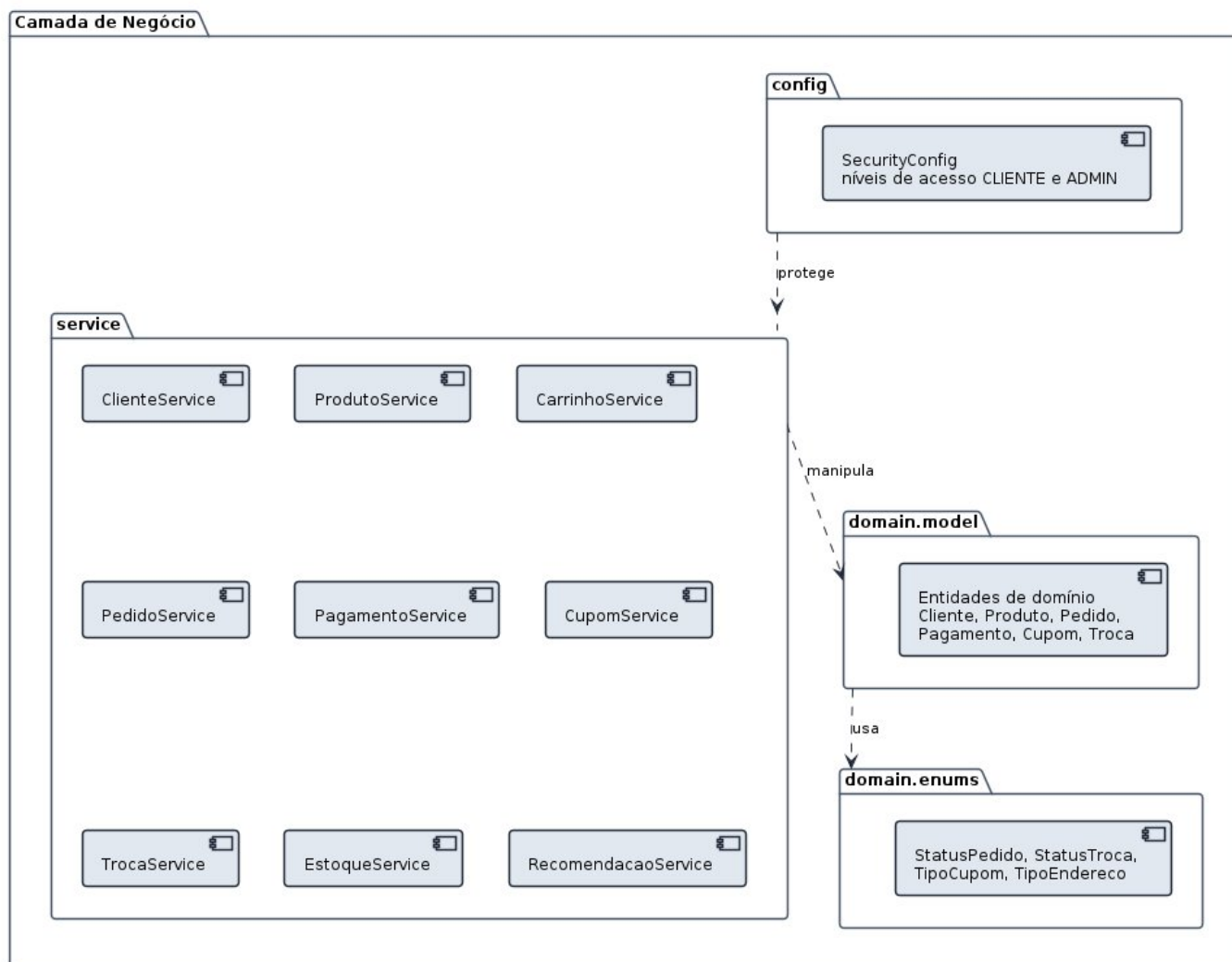


Figura 7 – Pacotes da camada de negócio

9.2.1. Pacote Controller

A Figura 8 apresenta as principais classes de controle do sistema e as operações que cada uma expõe na API. Todas seguem o mesmo padrão: recebem o objeto de entrada, delegam ao serviço correspondente e devolvem o objeto de resposta.

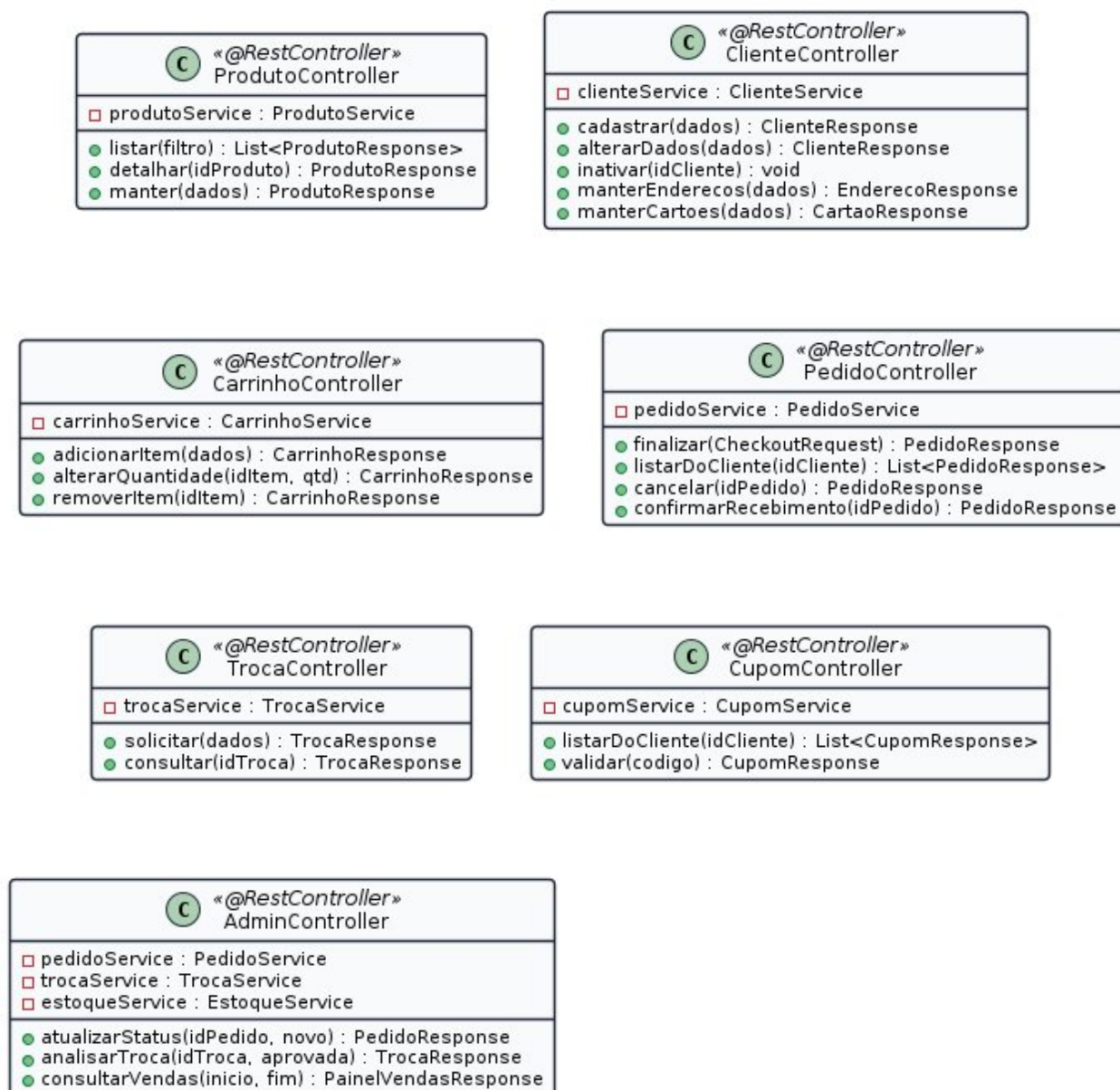


Figura 8 – Principais classes de controle

9.2.2. Pacote Model

As Figuras 9 e 10 apresentam as principais classes de domínio. A primeira reúne as classes de cadastro, catálogo e estoque; a segunda, as classes envolvidas na venda, no pagamento e na troca. Os atributos e as operações mostrados são preliminares e serão refinados durante a implementação.

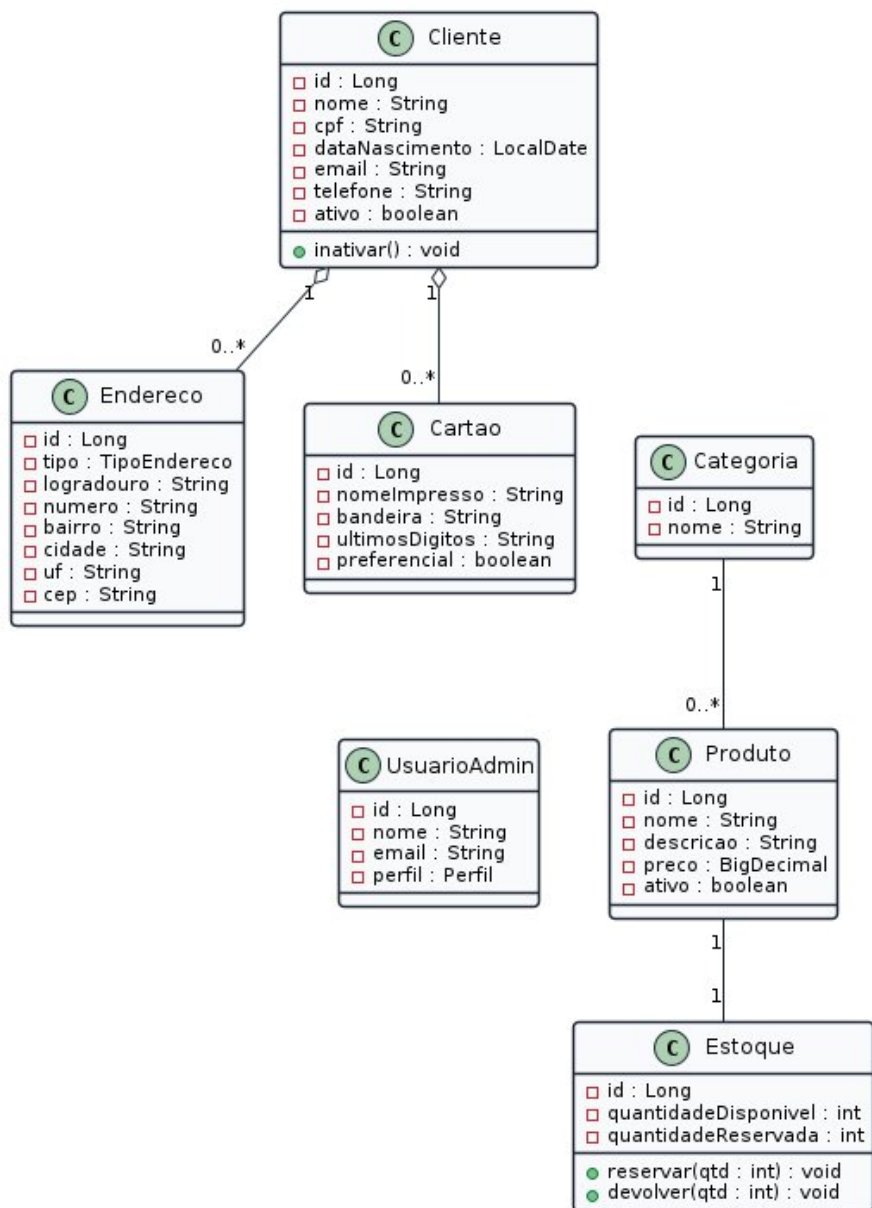


Figura 9 – Classes de domínio: clientes, produtos e estoque

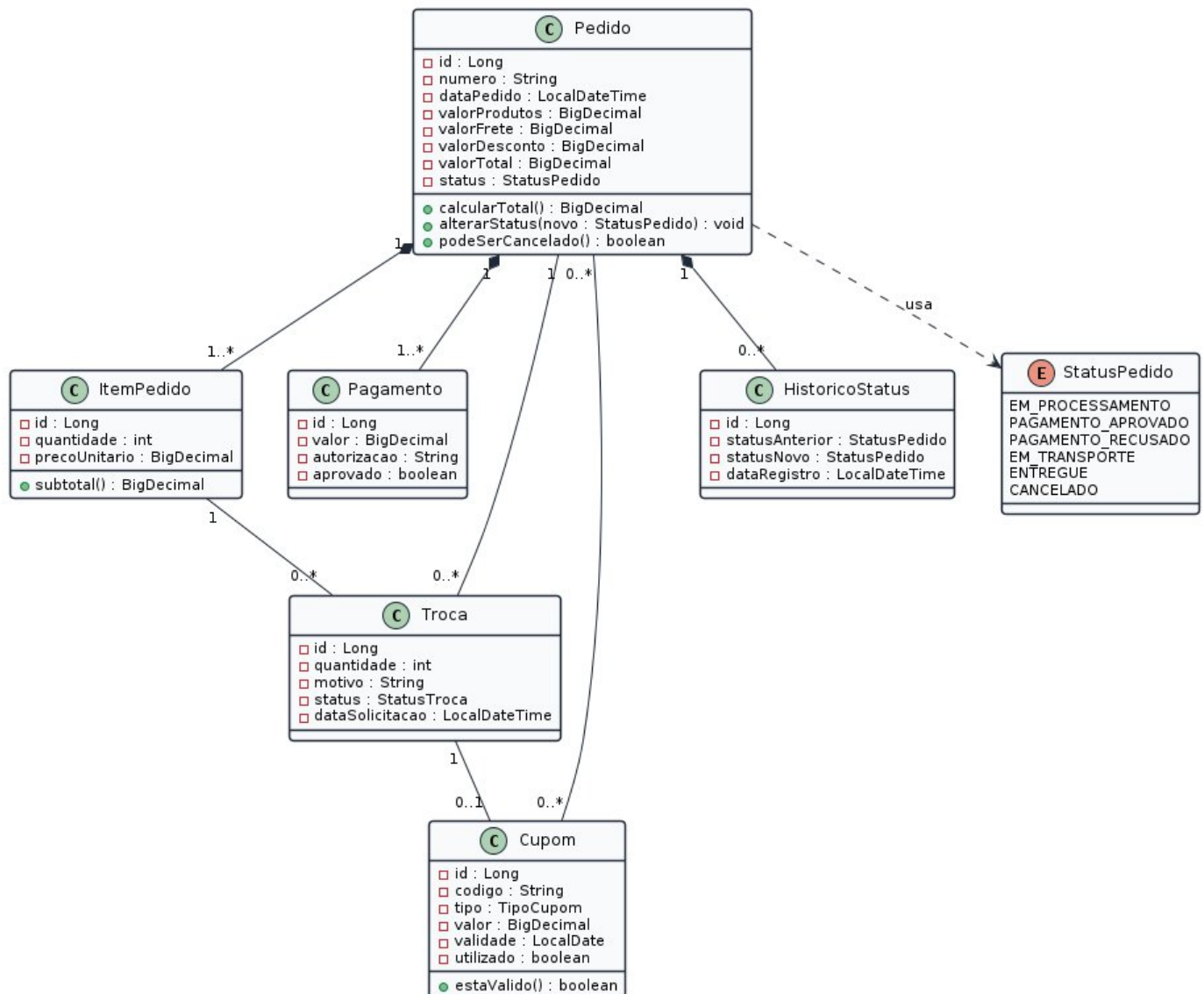


Figura 10 – Classes de domínio: pedidos, pagamentos, cupons e trocas

9.3. Camada de Persistência

A persistência é feita com Spring Data JPA sobre o Hibernate. O pacote repository contém uma interface por entidade principal, todas estendendo JpaRepository, o que fornece as operações básicas de gravação e consulta sem implementação manual; consultas específicas são declaradas por convenção de nome ou com a anotação @Query. O Hibernate traduz essas operações para o SQL do PostgreSQL, e o acesso ao banco é feito pelo driver JDBC configurado em application.properties. A Figura 11 mostra a organização dessa camada.

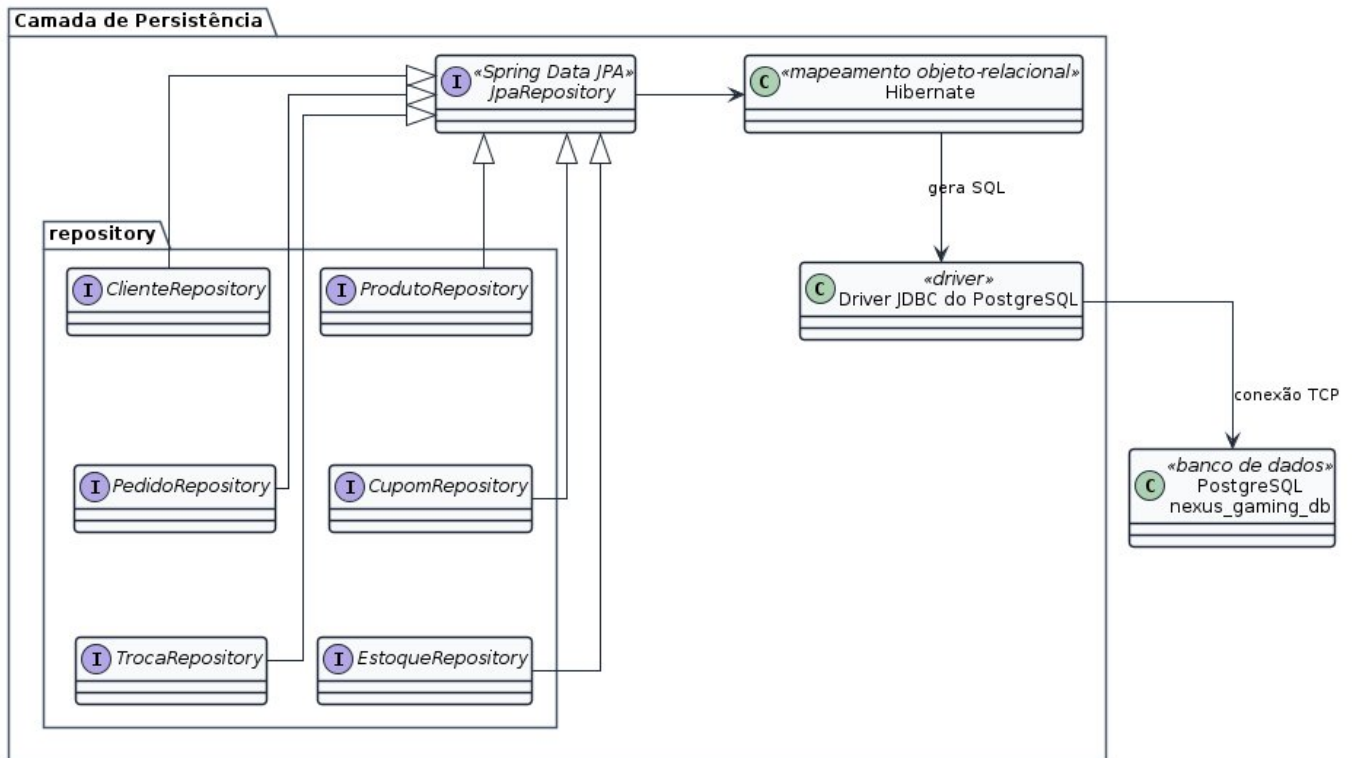


Figura 11 – Camada de persistência

9.4. Realização dos Casos de Uso Significativos

A Figura 12 mostra como as classes das diferentes camadas colaboram para realizar o caso de uso Finalizar pedido, o mais significativo do sistema. A requisição entra pelo controlador; o serviço de pedido coordena a validação dos cupons, a reserva do estoque e o processamento do pagamento; e só então o pedido é persistido com o status resultante.

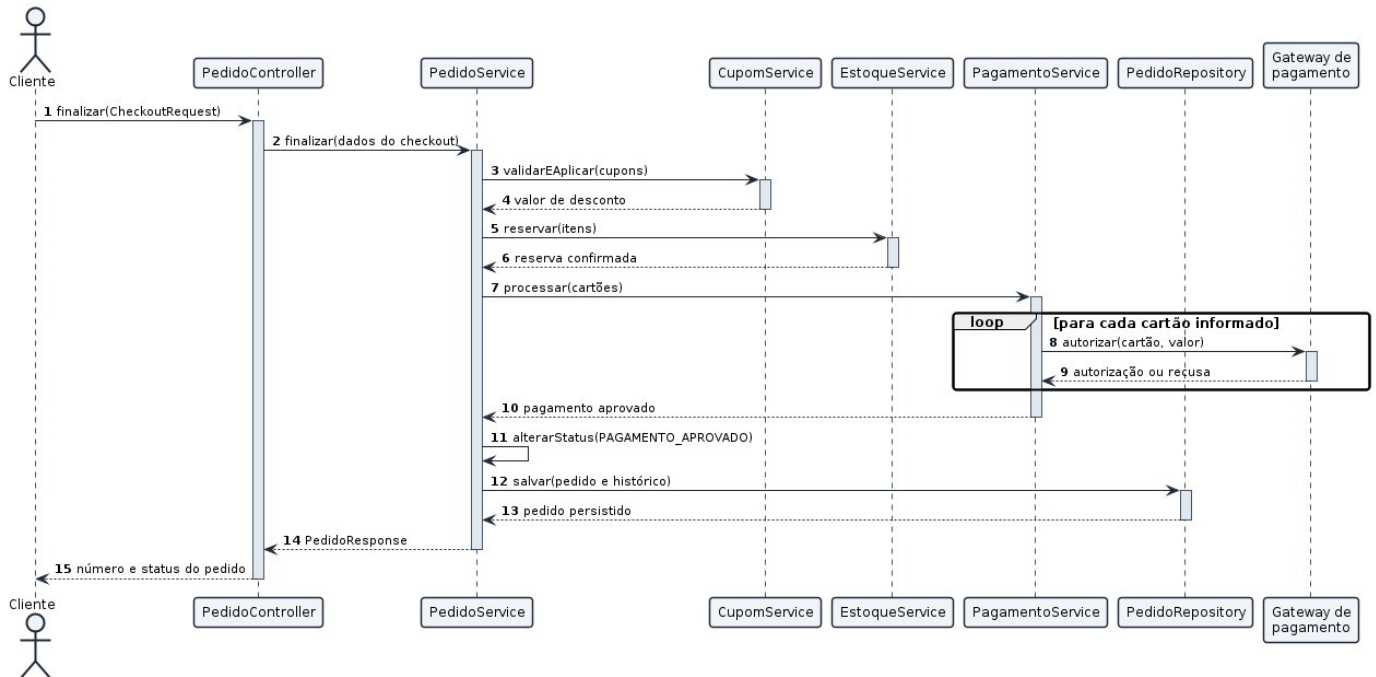


Figura 12 – Realização do caso de uso Finalizar pedido (diagrama de sequência)

A sequência evidencia duas decisões de arquitetura. A primeira é que o serviço de pedido é o único ponto que coordena a operação completa, o que mantém em um só lugar a regra sobre o que fazer quando uma das etapas falha. A segunda é que o acesso ao gateway de pagamento ocorre pela camada de integração, sem que as demais classes conheçam o provedor contratado.

10. Visão de Implantação

Esta seção descreve o ambiente em que o Nexus Gaming Gear é executado e como seus componentes se comunicam.

Durante o desenvolvimento, todos os componentes rodam na máquina de cada integrante: a aplicação sobe pelo Maven, com o Tomcat embutido na porta 8080, e o PostgreSQL responde na porta 5432 do mesmo computador. Para a entrega, prevê-se a separação entre o servidor de aplicação e o servidor de banco de dados, conforme a Figura 13.

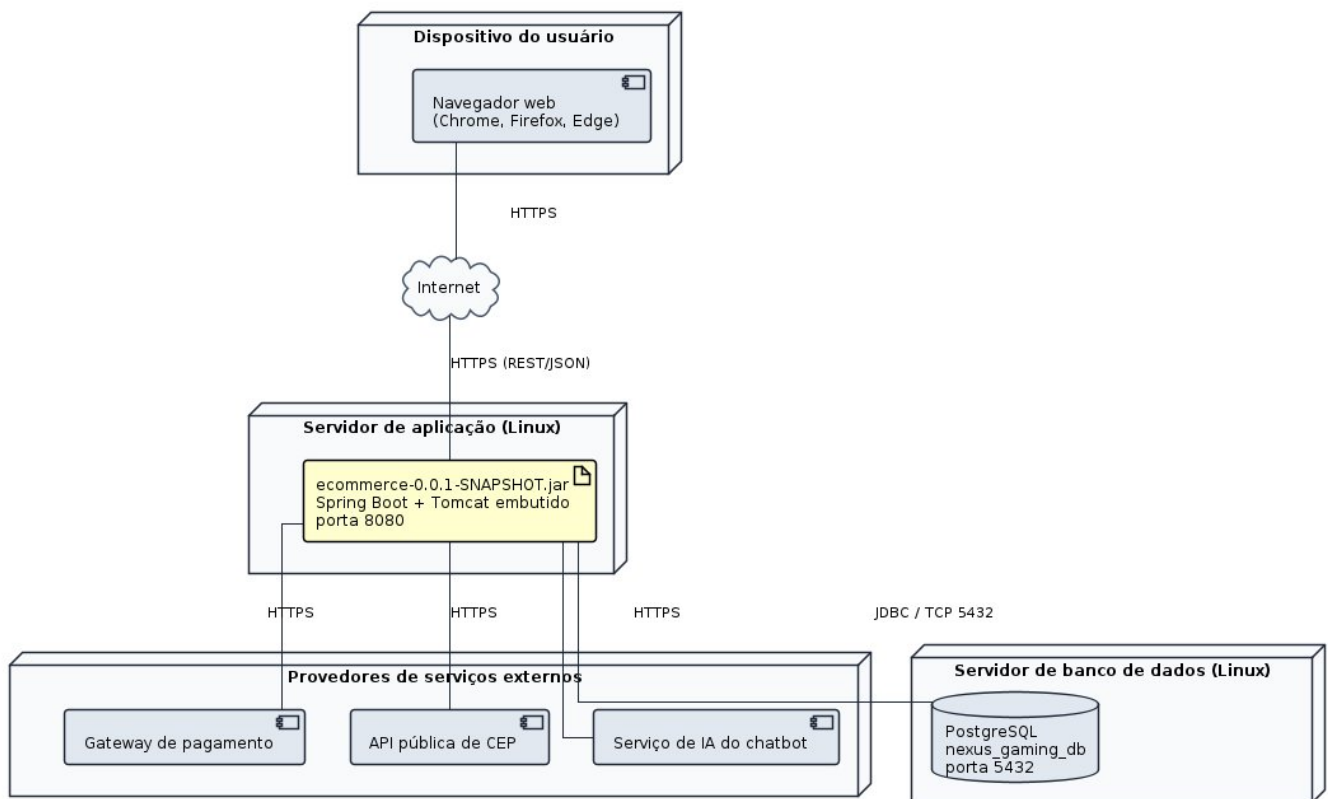


Figura 13 – Visão de implantação do Nexus Gaming Gear

Na Figura 13 observam-se os seguintes nós:

- **Dispositivo do usuário:** computador ou celular com navegador (Chrome, Firefox ou Edge), onde a interface é executada e de onde partem as chamadas à API;
- **Servidor de aplicação:** máquina Linux que executa o artefato ecommerce.jar, gerado pelo Maven, contendo a aplicação Spring Boot e o Tomcat embutido na porta 8080;
- **Servidor de banco de dados:** máquina Linux com o PostgreSQL e a base nexus_gaming_db, acessada pelo servidor de aplicação via JDBC na porta 5432;
- **Provedores de serviços externos:** gateway de pagamento, API pública de consulta de CEP e serviço de inteligência artificial do assistente virtual, todos acessados por HTTPS a partir do servidor de aplicação.

A comunicação entre o navegador e o servidor de aplicação é feita por HTTPS. Em ambiente de desenvolvimento, navegador, aplicação e banco coincidem na mesma máquina; a separação mostrada na figura vale para o ambiente de entrega.

11. Visão de Implementação

Esta visão descreve como o código do sistema está organizado em diretórios e módulos de implementação.

A estrutura do projeto segue diretamente as camadas apresentadas na Visão de Lógica: dentro de src/main/java, o pacote raiz com.nexusgaming.ecommerce se subdivide em config, controller, dto, service,



domain (com model e enums), repository, integration e exception. Em src/main/resources ficam o arquivo application.properties, com a configuração de acesso ao banco, e o diretório static, com o front-end. Os testes automatizados ficam em src/test/java, espelhando a mesma divisão de pacotes. Como a implementação reproduz a organização lógica já descrita, não são necessários diagramas adicionais nesta seção.

12. Visão de Dados

A persistência do Nexus Gaming Gear utiliza o banco de dados relacional PostgreSQL, na base nexus_gaming_db, com mapeamento objeto-relacional feito pelo Hibernate por meio do Spring Data JPA. O controle de transações é declarativo, com a anotação @Transactional nas classes de serviço, de modo que operações compostas, como o fechamento de um pedido, sejam confirmadas ou desfeitas por inteiro.

A Figura 14 apresenta o modelo lógico de dados e as Figuras 15 e 16, o modelo físico, dividido em dois recortes para facilitar a leitura: cadastro, catálogo e estoque; e vendas, pagamentos e trocas.

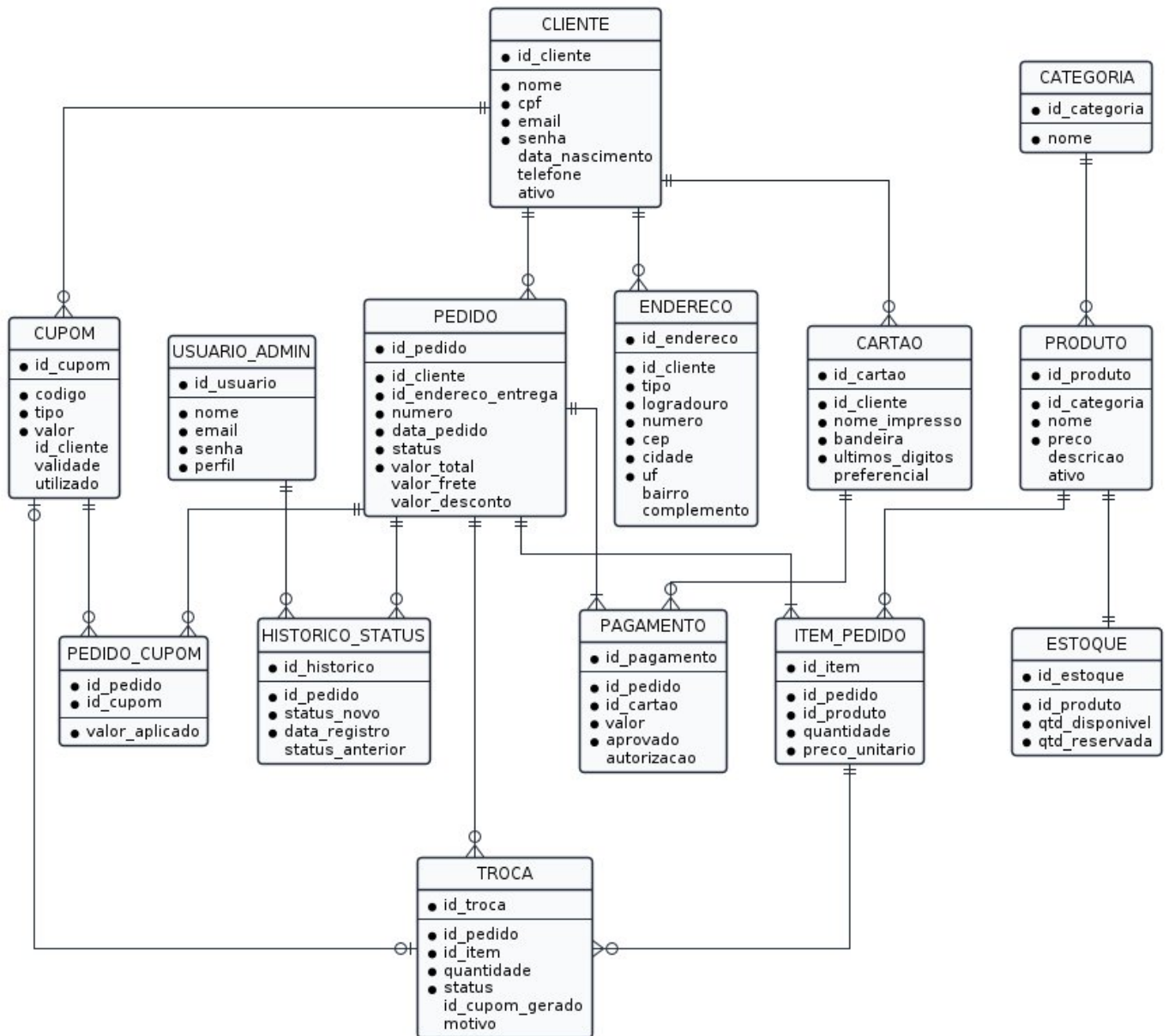


Figura 14 – Modelo lógico de dados (DER)

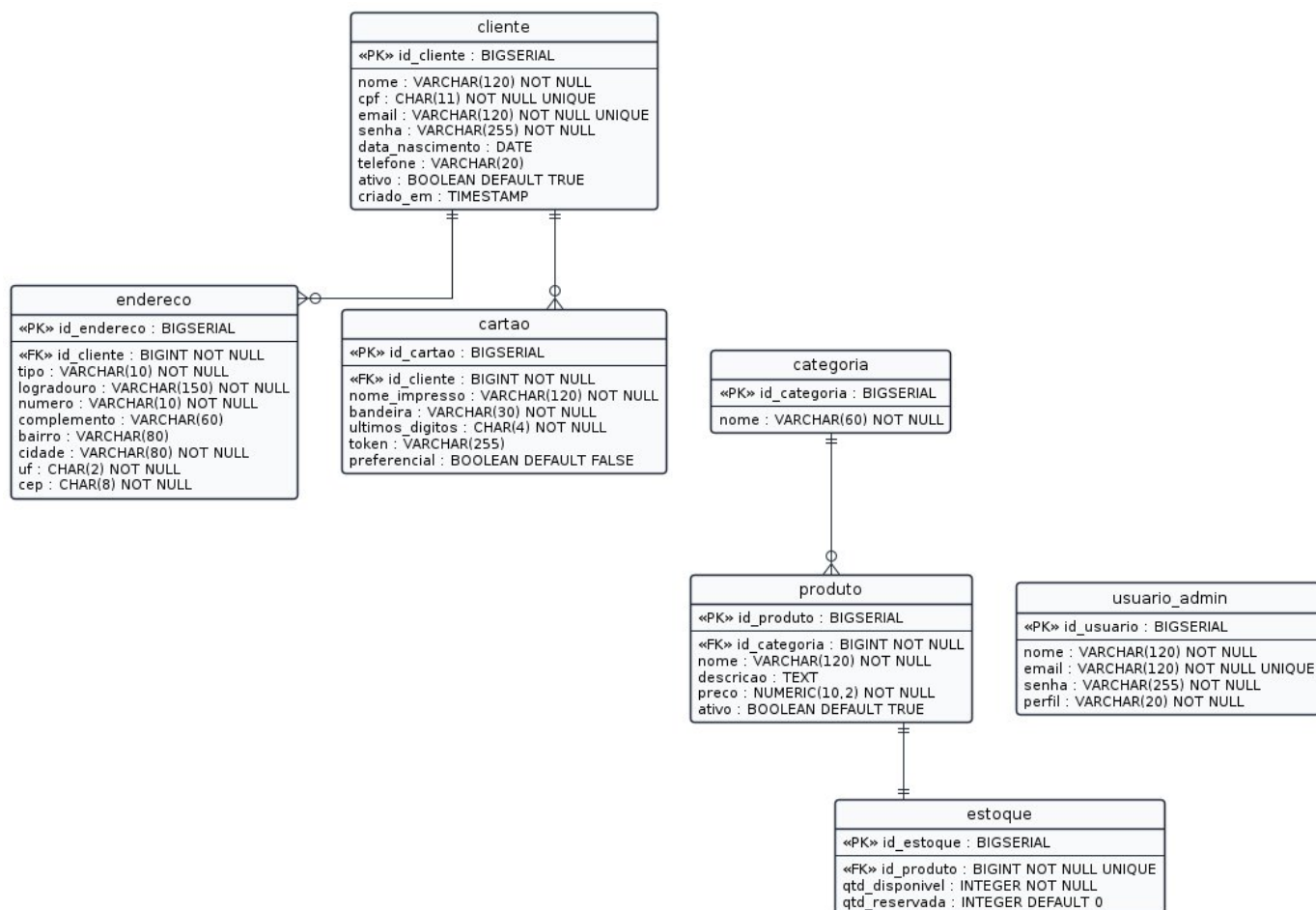


Figura 15 – Modelo físico: cadastro, catálogo e estoque

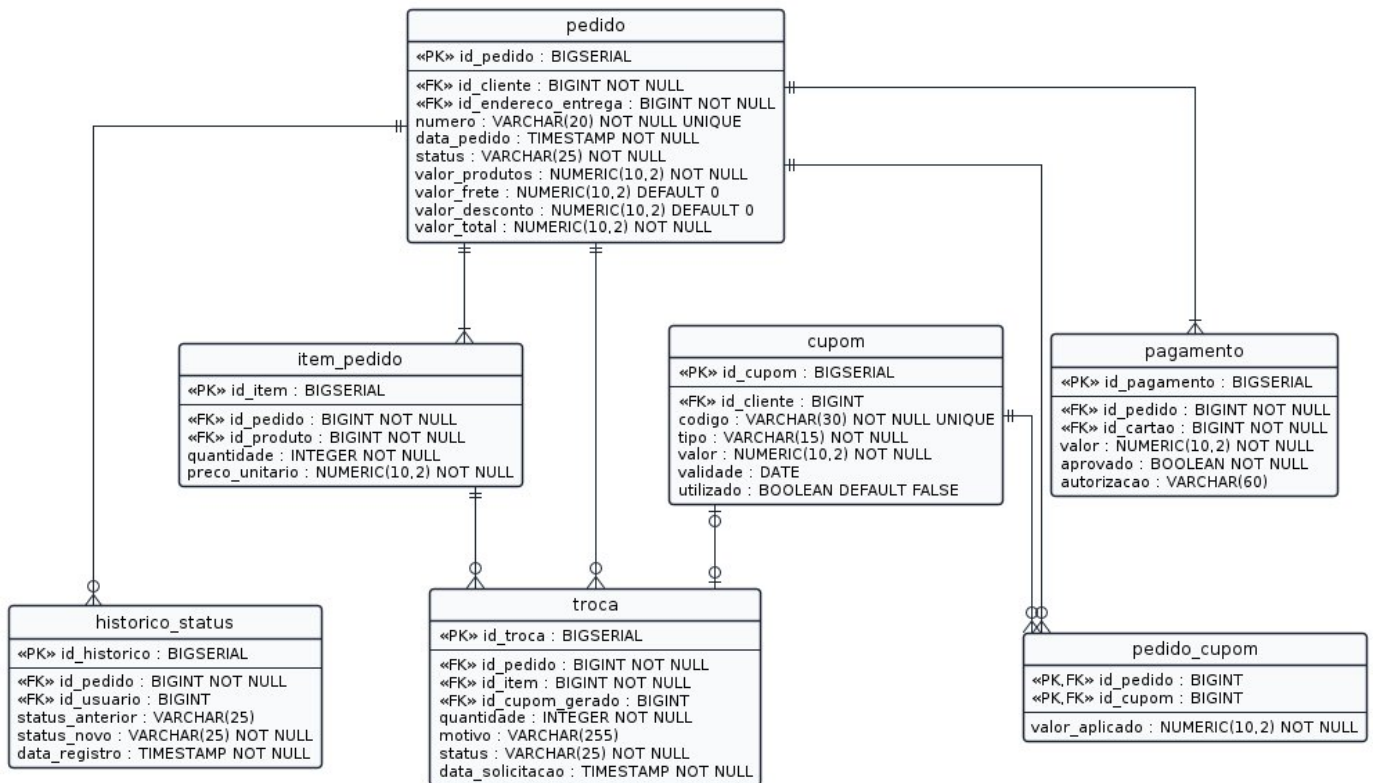


Figura 16 – Modelo físico: pedidos, pagamentos, cupons e trocas

A Tabela 1 exemplifica o mapeamento entre as classes de domínio e as tabelas correspondentes do banco de dados.

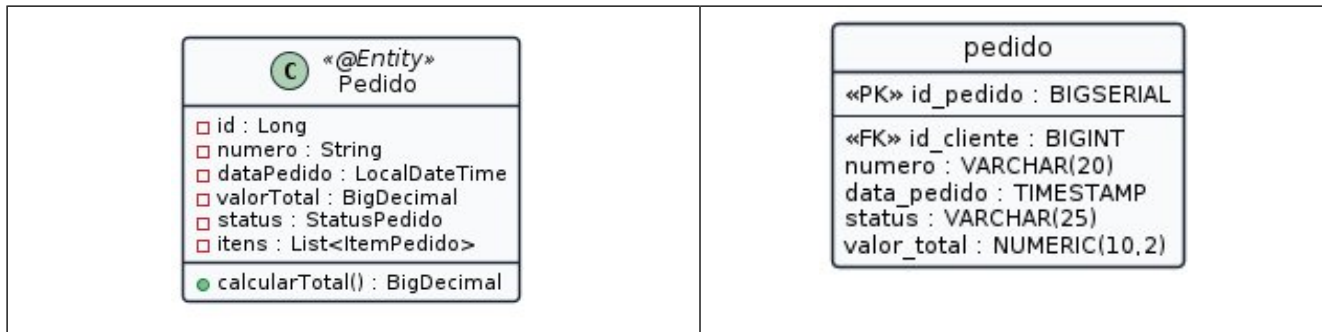
O mapeamento é declarado por anotações JPA nas próprias classes de domínio (@Entity, @Table, @Column e as anotações de relacionamento), sem arquivos de configuração externos.

Os identificadores são gerados pelo banco, em colunas do tipo BIGSERIAL, e mapeados pela estratégia IDENTITY.

Durante o desenvolvimento o esquema é criado e atualizado automaticamente pelo Hibernate (propriedade spring.jpa.hibernate.ddl-auto=update); para a entrega será gerado um script SQL de criação das tabelas, versionado junto com o código.

Tabela 1 – Exemplo de mapeamento objeto-relacional

Classe de domínio	Tabela no banco de dados
<pre> classDiagram class Cliente { «@Entity» id : Long nome : String cpf : String email : String ativo : boolean enderecos : List<Endereco> inativar() : void } </pre>	<pre> table cliente { «PK» id_cliente : BIGSERIAL nome : VARCHAR(120) cpf : CHAR(11) email : VARCHAR(120) ativo : BOOLEAN } </pre>



13. Tamanho e Performance

O Nexus Gaming Gear é dimensionado para uma loja especializada de pequeno porte. A estimativa inicial da equipe é de um catálogo entre 200 e 500 produtos ativos, uma base de 2.000 a 5.000 clientes cadastrados no primeiro ano e um volume entre 20 e 50 pedidos por dia, o que representa cerca de 1.500 pedidos por mês e um crescimento de poucas centenas de megabytes por ano na base de dados, já considerando o histórico de status e as trocas.

Os picos de utilização concentram-se em datas comerciais e em lançamentos de produtos, quando o número de acessos simultâneos pode ser várias vezes maior que a média. A operação mais frequente é a consulta ao catálogo, executada inclusive por visitantes não autenticados, e por isso deve responder em até três segundos. Já o fechamento do pedido é a operação mais lenta, por depender da autorização externa dos cartões, e precisa tratar demora e indisponibilidade do gateway sem deixar o cliente sem resposta.

No contexto da disciplina, os testes serão feitos com massa de dados fictícia, em volume reduzido e em ambiente local. As estimativas acima orientam decisões de projeto como o uso de paginação nas consultas ao catálogo e a criação de índices para as colunas usadas nas buscas por cliente e por status do pedido.

14. Qualidade

Por se tratar de um sistema de comércio eletrônico, os atributos de qualidade mais críticos para o Nexus Gaming Gear são a integridade das informações de pedidos, pagamentos e estoque e a segurança dos dados dos clientes.

A **integridade** é crítica porque um mesmo pedido envolve reserva de estoque, aplicação de cupons e várias autorizações de pagamento. Uma falha no meio dessa operação pode deixar estoque reservado sem pedido correspondente, cupom marcado como usado sem desconto aplicado ou pagamento autorizado sem registro no sistema. São situações que geram prejuízo direto e desgaste com o cliente. Por isso o fechamento do pedido é tratado como uma transação única e toda mudança de status fica registrada em histórico, permitindo auditar o que aconteceu com cada pedido.

A **segurança** é crítica porque o sistema guarda dados pessoais (nome, CPF, endereços e telefone) e lida com meios de pagamento. As senhas são armazenadas cifradas; o número completo do cartão não é persistido, ficando no banco apenas bandeira, nome impresso e os quatro últimos dígitos, com a guarda dos dados sensíveis a cargo do gateway; e as operações da página de administração só são aceitas de contas com nível de acesso administrativo. Como a aplicação fica exposta na internet, toda entrada recebida pela API é validada no servidor, e não apenas no navegador.

A **disponibilidade** também importa, ainda que em grau menor no contexto acadêmico: loja fora do ar significa venda perdida. Por isso o assistente virtual foi projetado para falhar de forma isolada, sem afetar a navegação, e a indisponibilidade do gateway de pagamento deixa o pedido em situação pendente em vez de



descartá-lo. Os critérios de aceitação relacionados a esses atributos serão detalhados nos requisitos não funcionais do Documento de Requisitos.

15. Cronograma Macro

A tabela a seguir apresenta os marcos previstos para o projeto no semestre 2026/2. As datas acompanham o calendário da disciplina e devem ser confirmadas com o professor.

Marco	Data prevista
Definição do tema e protótipo da interface	Concluído – agosto/2026
Entrega do Documento de Visão de Projeto	19/09/2026
Entrega do Documento de Requisitos	10/10/2026
Modelagem de dados e criação do banco nexus_gaming_db	17/10/2026
Sprint 1 – cadastro de clientes, catálogo e estoque	31/10/2026
Sprint 2 – carrinho, fechamento de pedido e pagamento	21/11/2026
Sprint 3 – trocas, cupons e página de administração	05/12/2026
Testes integrados e correções	12/12/2026
Apresentação final do sistema	19/12/2026

Obs.: os prazos apresentados são uma estimativa inicial, feita com as informações disponíveis nesta etapa. O cronograma detalhado, com a distribuição das atividades entre os integrantes, será elaborado na fase de planejamento e pode alterar estas datas.



16. Referências

UML – Unified Modeling Language. Object Management Group. Disponível em: <https://www.omg.org/spec/UML/>

Spring Boot Reference Documentation. Disponível em: <https://docs.spring.io/spring-boot/>

Spring Data JPA Reference Documentation. Disponível em: <https://spring.io/projects/spring-data-jpa>

PostgreSQL Documentation. Disponível em: <https://www.postgresql.org/docs/>

Repositório do projeto Nexus Gaming Gear. Disponível em: <https://github.com/lucasscarmona/nexus-gaming-ecommerce>

Material didático e orientações apresentadas em aula na disciplina.